

SDLC Models – When to Use, Advantages, and Disadvantages

Waterfall Model

When to Use

- Requirements are very clear, stable, and well documented.
- Technology and tools are well understood.
- Project is small or medium with minimal expected changes.
- Customer requirements are unlikely to change during development.

Advantages

- Simple and easy to understand and manage.
- Clearly defined stages and milestones.
- Documentation is produced at each phase which helps maintenance.
- Good for projects with fixed scope and well-known requirements.

Disadvantages

- Very difficult to accommodate requirement changes.
- Working software is not available until late in development.
- Risk of building the wrong product if requirements were misunderstood.
- Not suitable for complex or long-term projects.

V-Model (Verification and Validation)

When to Use

- Requirements are well defined and stable.
- Project is small to medium sized.
- Quality assurance and testing are extremely important.
- Used in safety-critical systems such as medical or embedded systems.

Advantages

- Testing activities are planned early in the development cycle.
- Each development phase has a corresponding testing phase.
- Errors can be detected earlier compared to waterfall.
- Improves product reliability and quality.

Disadvantages

- Very rigid and inflexible.

- Changes are difficult to implement once development starts.
- Not suitable for projects with evolving requirements.
- Working software appears late in the process.

Incremental Model

When to Use

- Requirements are known but system needs to be delivered quickly.
- Large systems that can be divided into smaller modules.
- Customers want partial functionality early.
- When staffing is limited for full development at once.

Advantages

- Working software delivered early.
- Easy to test and debug smaller modules.
- Customer feedback can be incorporated between increments.
- Lower initial delivery cost and reduced project risk.

Disadvantages

- Requires good system architecture planning.
- Integration between increments may become complex.
- Not suitable when system cannot be modularized.
- Requires careful planning and management.

Prototyping Model

When to Use

- Customer requirements are unclear or incomplete.
- User interface intensive applications.
- When developers need to explore technical feasibility.
- When user feedback is required early.

Advantages

- Helps understand user requirements better.
- Users can see and interact with early version of system.
- Early detection of requirement errors.
- Improves communication between users and developers.

Disadvantages

- Users may mistake prototype for final product.
- May lead to poor design if prototype is directly converted.

- Development cost and time may increase.
- Prototype may ignore quality and performance considerations.

Spiral Model

When to Use

- Large and complex projects.
- High-risk projects requiring continuous risk analysis.
- When requirements are unclear and evolving.
- Projects involving new technology.

Advantages

- Strong focus on risk analysis and management.
- Iterative development allows gradual improvement.
- Customer feedback at every stage.
- Early detection and mitigation of risks.

Disadvantages

- Expensive due to extensive risk analysis.
- Requires highly skilled experts for risk assessment.
- Complex to manage.
- Not suitable for small projects.

RAD Model (Rapid Application Development)

When to Use

- Project needs to be developed quickly (2–3 months).
- System can be modularized.
- High availability of skilled developers.
- When tools for automatic code generation are available.

Advantages

- Very fast development cycle.
- Encourages component reuse.
- Customer feedback obtained early.
- Integration problems reduced due to parallel development.

Disadvantages

- Requires highly skilled developers.
- Not suitable for large or complex systems.
- Not suitable if system cannot be modularized.

- High dependency on modeling tools.

Component-Based Development

When to Use

- When reusable components or COTS products are available.
- When reducing development time is critical.
- For large enterprise systems using modular architecture.

Advantages

- Reduces development time.
- Lower project cost.
- Encourages software reuse.
- Improves reliability because components are pre-tested.

Disadvantages

- Integration of components can be difficult.
- Dependency on third-party components.
- Customization may be limited.
- Compatibility issues between components.

Agile Model

When to Use

- When requirements frequently change.
- When close collaboration with customers is possible.
- For projects requiring rapid delivery of working software.
- Small to medium sized teams with strong communication.

Advantages

- Highly flexible and adaptable to changes.
- Continuous delivery of working software.
- Strong customer involvement.
- Faster detection of problems through iterative development.

Disadvantages

- Requires experienced and disciplined teams.
- Documentation may be minimal.
- Not ideal for very large or safety-critical systems.
- Difficult to estimate cost and time early.